Universidad
Manuela Beltrán
Acreditada en Alta Calidad Multicampus

GUÍA 7 · PRUEBAS DE SEGURIDAD AUTOMATIZADAS

Análisis estático de seguridad (SAST) con gosec

Un escáner de código fuente para Go, probado sobre una aplicación vulnerable y sobre código propio.



Informe HTML generado por gosec sobre GoVWA (captura real).

Cobos, Dilan Steven
Rodríguez Ángel, Santiago
Alarcón Ospina, Víctor Alfonso

Universidad Manuela Beltrán · Ingeniería de Software · 2026

HERRAMIENTA SAST PARA GO

¿Qué es gosec?

SAST: revisa el código fuente sin ejecutar la aplicación, como revisar los planos antes de construir.

Código abierto, licencia Apache 2.0.

Mantenido por el proyecto securego.

Requiere Go 1.25 o superior.

Reglas con su CWE y filtros por severidad y confianza.

Limitación clave: solo analiza código Go.

62 reglas

Credenciales, SQL, comandos, criptografía, TLS, archivos, cookies y servidores HTTP.

Flujo de datos

Sigue la entrada del usuario hasta SQL, comandos, rutas, SSRF, XSS y logs (G701 a G710).

9 formatos

text, json, yaml, csv, junit-xml, html, sonarqube, golint y sarif.

Listo para CI

GitHub Action oficial (securego/gosec) y filtros para fallar el pipeline.

Configuración y entorno

1. **Objetivo**
   GoVWA: aplicación web vulnerable escrita en Go (20 archivos, 1.569 líneas) y código propio.
2. **Entorno**
   Windows 11, Go 1.27.2. Sin Docker, sin cuenta y sin administrador.
3. **Instalación**
   Un comando. Compila gosec desde el código fuente: 1 min 50 s.
4. **Versión real**
   El binario dice «dev»; go version -m revela la v2.30.0.
5. **Escaneo**
   Un análisis completo de GoVWA tarda unos 5 segundos.

COMANDOS USADOS

```
# instalar
go install github.com/securego/gosec/v2/cmd/gosec@latest

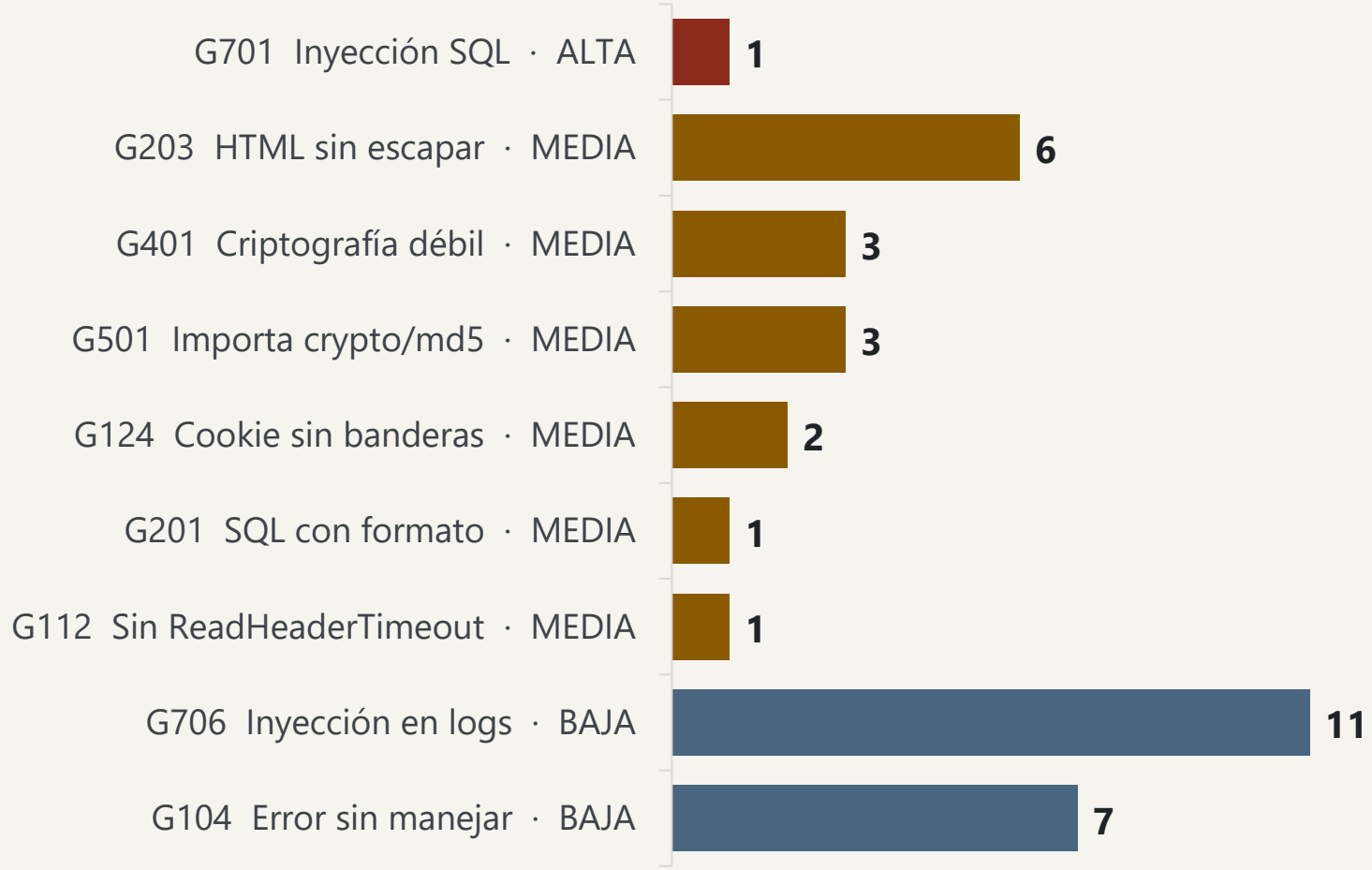
# comprobar la versión real
go version -m gosec.exe

# escanear y guardar el informe
gosec -fmt=json -out=govwa.json ./...
gosec -fmt=html -out=govwa.html ./...
```

Objetivo fijado: GoVWA, commit 4058f79. Cada resultado se puede repetir con scripts/escanear.sh.

RESULTADOS SOBRE GOVWA

35 hallazgos en 20 archivos



1 **Severidad ALTA**
inyección SQL

16 **Severidad MEDIA**
HTML, cripto, cookies, SQL

18 **Severidad BAJA**
logs y errores sin manejar

Confianza: 28 alta y 7 baja. La baja se revisa a mano.

HALLAZGO 1 · EL ÚNICO DE SEVERIDAD ALTA

Inyección SQL

GOSEC MARCÓ · vulnerability/sqli/function.go

```
37  getProfileSql := fmt.Sprintf(`SELECT p.user_id, p.full_name,
38      p.city, p.phone_number FROM Profile as p, Users as u
39      where p.user_id = u.id
40      and u.id=%s`, uid) //here is the vulnerable query
41  rows, err := DB.Query(getProfileSql)
```

CORRECCIÓN DEL PROPIO REPOSITORIO · SafeQueryGetData

```
66  and u.id=?`
69  stmt, err := DB.Prepare(getProfileSql) //prepare statement
75  err = stmt.QueryRow(uid).Scan(&p.Uid, &p.Name, ...)
```

Control: la versión segura, en el mismo archivo, no fue marcada.

SEVERIDAD ALTA · CONFIANZA ALTA

G701 y G201 · CWE-89

Riesgo

El identificador del usuario se pega dentro de la instrucción SQL.

Un atacante cambia su significado y lee o altera datos de otros usuarios.

Mitigación

Consulta parametrizada: el dato viaja aparte (?).

Validar el tipo: el id debe ser numérico.

HALLAZGO 2 · SEVERIDAD MEDIA, CONFIANZA ALTA

Contraseñas protegidas con MD5

GOSEC MARCÓ · user/user.go

```
 88  pass := Md5Sum(r.FormValue("password"))
159  func Md5Sum(text string) string {
160      hasher := md5.New()
```

NUESTRA CORRECCIÓN · ejemplo-propio/corregido

```
h, err := bcrypt.GenerateFromPassword(
    []byte(clave), bcrypt.DefaultCost)
// al iniciar sesión:
// bcrypt.CompareHashAndPassword(h, ...)
```

La misma debilidad en 3 archivos: 6 hallazgos (user.go, idor.go y csa.go).

SEVERIDAD MEDIA **CONFIANZA ALTA**

G401 y G501 · CWE-328 y CWE-327

Riesgo

MD5 es muy rápido y aquí no lleva sal.

Con una base de datos filtrada, las contraseñas comunes se recuperan en segundos.

Mitigación

bcrypt o argon2id: lentos a propósito y con sal.

Migrar los hashes viejos al iniciar sesión.

HALLAZGO 3 · CONFIANZA BAJA: REQUIERE REVISIÓN HUMANA

HTML sin escapar (XSS)

GOSEC MARCÓ · vulnerability/xss/xss.go

```
39  term := r.FormValue("term")
61  vuln := fmt.Sprintf("<b>%s</b>", term)
62  data["value"] = template.HTML(value)
63  data["term"] = template.HTML(vuln)
```

NUESTRA CORRECCIÓN

```
data["term"] = term   // html/template lo escapa solo
// HTML propio: solo constantes, o sanitizado
```

6 hallazgos G203 en 2 archivos. gosec duda (confianza baja): aquí sí son reales.

SEVERIDAD MEDIA | CONFIANZA BAJA

G203 · CWE-79

Riesgo

template.HTML le dice a Go que el texto ya es seguro y salta el escapado automático.

Un término con una etiqueta de script se ejecutaría en el navegador de la víctima.

Mitigación

Pasar texto plano: html/template escapa solo.

Si hace falta HTML, sanitizarlo y añadir una política CSP.

EL CICLO COMPLETO: DETECTAR, CORREGIR, VOLVER A ESCANEAR

Antes y después: código propio

17

hallazgos en el código vulnerable (6 altos, 9 medios y 2 bajos)

↓

0

hallazgos activos tras corregir

(2 suprimidos con #nosec y justificación escrita)

Error (regla de gosec)	Corrección aplicada
SQL armado a mano (G201, G701)	Consulta parametrizada: db.Query(sql, id)
Contraseñas con MD5 (G401, G501)	bcrypt con sal y costo
Credencial en el código (G101)	Se lee con os.Getenv, nunca del fuente
TLS sin verificar (G402)	Verificación activa y TLS 1.2 como mínimo
Tokens con math/rand (G404)	crypto/rand
Cookie sin banderas (G124)	Secure, HttpOnly y SameSite
Servidor sin límites (G114)	http.Server con tiempos límite, solo en 127.0.0.1
Ruta del usuario (G304, G703)	os.OpenRoot: no puede salir de la carpeta
Comando del usuario (G204, G702, G705)	Lista de permitidos, sin shell ni eco de salida

LÍMITES, CON EVIDENCIA

Lo que gosec no vio

IDOR

Es un fallo de lógica. En idor.go gosec marcó el MD5 y errores sin manejar, pero no el acceso a datos de otro usuario.

Secretos en config.json

La contraseña de la base y la clave de sesión están en un JSON. gosec solo analiza archivos .go.

Credenciales: depende del nombre

Con 4 variables con credenciales, G101 marcó 1. password = "admin123" exige ignore_entropy y las variables llamadas clave... nunca.

Falsos positivos

Tras validar con lista de permitidos, G204 y G702 siguen avisando. Se suprimen con #nosec y justificación; gosec lo registra.

Por eso SAST se complementa con DAST, SCA y revisión manual.

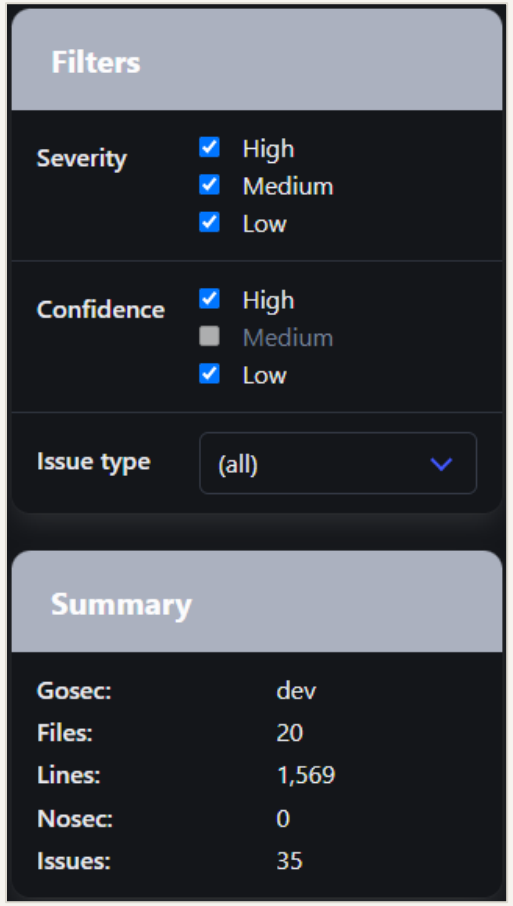
VALORACIÓN DEL GRUPO

Experiencia de uso y conclusiones

Criterio	Nuestra experiencia
Instalación	Fácil: un comando (go install), unos 2 min compilando. Requiere Go; no necesita Docker.
Facilidad de uso	Muy fácil: gosec ./... y listo. Un análisis completo de GoVWA tarda unos 5 s.
Resultados	Claros: regla, CWE, severidad, confianza, línea y código. Con falsos positivos y negativos.
Interfaz	Terminal e informe HTML con filtros; JSON, SARIF y JUnit para CI.

Conclusiones

En 5 s encontró un SQLi real y 34 hallazgos más; corregir el código los llevó a 0.
Úsalo en CI (Action oficial) y atiende primero severidad alta con confianza alta.
No reemplaza DAST ni la revisión: no vio el IDOR ni los secretos de config.json.



Informe HTML de gosec (captura real)

Preguntas y dudas

Todo es reproducible: carpeta Guia 7/Gosec del repositorio.

Scripts, resultados reales, código propio y esta presentación.